Phần mềm Quản Lý Nhà Thuốc Minh Giang (Microservices)

Dự án phát triển Hệ thống quản lý nhà thuốc Minh Giang theo kiến trúc Microservices — môn học SOA 2026.

Kiến trúc tổng quan

```
Browser / POS / Admin
        │
        ▼
  API Gateway :8000   ←── JWT xác thực tập trung
        │
  ┌─────┼──────────────────────────────────┐
  │     │                                  │
  ▼     ▼         ▼          ▼             ▼
8001  8002       8003        8004          8005
identity catalog  order       cms        notification
service  service  service   service       service
  │        │        │          │             │
  └────────┴────────┴──────────┴─────────────┘
                   │
               MySQL 8.0  (mỗi service 1 schema riêng)
               RabbitMQ   (async events — TODO)
```

Service	Port	Database schema	Mô tả
api-gateway	8000	—	Proxy + JWT auth
identity-service	8001	`mg_identity`	User, Customer, Role, Shift
catalog-service	8002	`mg_catalog`	Product, Category, Batch, Inventory, Supplier, Location
order-service	8003	`mg_order`	Cart, Checkout, Order, Return
cms-service	8004	`mg_cms`	Article, Banner, Disease, Promotion, StoreConfig
notification-service	8005	`mg_notification`	Email (nodemailer), SMS stub, Template

Yêu cầu môi trường

- **Docker Desktop** (Windows / macOS) — bắt buộc
- Node.js ≥ 18 — chỉ cần nếu chạy service riêng lẻ (không bắt buộc khi dùng Docker)

- Git

Quick Start — Chạy toàn bộ bằng Docker (khuyên dùng)

```
# 1. Clone về
git clone <repo-url>
cd "Minh Giang Pharmacy"

# 2. Tạo file .env từ template
cp .env.example .env
# Mở .env và điền JWT_SECRET (bắt buộc), SMTP_* (nếu muốn gửi email
thật)

# 3. Khởi động tất cả
docker-compose up -d

# 4. Xem log (Ctrl+C để thoát, containers vẫn chạy)
docker-compose logs -f api-gateway
```

> Lần đầu chạy Docker sẽ pull image và `npm install` — mất vài phút.
> Từ lần sau: `docker-compose up -d` là đủ, node_modules đã được cache.

Kiểm tra hoạt động:

```
GET http://localhost:8000/health          → Gateway status + service
URLs
GET http://localhost:8000/api/catalog/health  → Catalog service health
```

Quick Start — Chạy một service riêng lẻ (local dev)

Dùng khi muốn debug nhanh, không cần khởi động toàn bộ stack.

```
# Bước 1: Đảm bảo MySQL đang chạy (trong Docker)
docker-compose up -d mysql-db

# Bước 2: Cài dependencies
cd backend/catalog-service
npm install

# Bước 3: Tạo .env
cp .env.example .env
# Sửa DB_HOST=localhost (vì kết nối trực tiếp, không qua Docker network)

# Bước 4: Chạy
npm run dev
```

> Thay `catalog-service` bằng service bạn đang phụ trách.

Cấu trúc thư mục

```
├── backend/
│   ├── api-gateway/          # Express proxy + JWT middleware
│   ├── identity-service/     # User / Auth / Customer / Role / Shift
│   ├── catalog-service/      # Product / Category / Batch / Inventory /
Supplier / Location
│   ├── order-service/        # Cart / Checkout / Order / Return
│   ├── cms-service/          # Article / Banner / Disease / Promotion /
StoreConfig
│   └── notification-service/ # Email / SMS / Template
├── frontend/
│   ├── client/               # Website khách hàng
│   ├── admin/                # Trang quản trị
│   └── pos/                  # Màn hình bán hàng POS
├── infrastructure/
│   └── database/             # Toàn bộ SQL schema + migration +
security patches
├── docker-compose.yml
├── .env.example              # Template biến môi trường (commit lên
Git)
└── README.md
```

Mỗi service có cùng cấu trúc:

```
backend/<service-name>/
├── index.js                  # Entrypoint Express
├── package.json
├── Dockerfile
├── .env.example
├── db/pool.js                # MySQL connection pool
├── middlewares/gatewayAuth.js
└── routes/
    ├── index.js              # Mount tất cả sub-router
    └── <resource>/
        └── <resource>.routes.js
```

Luồng xác thực JWT

```
Client  ──POST /api/identity/auth/login──►  Gateway  ──►  identity-
service
```

```
          ◄── { token: "eyJ..." }
  ──────────────────────────────────────────

  Client  ──GET /api/order/orders  ──────►  Gateway
          Authorization: Bearer eyJ...       │ Verify JWT → req.user
                                             │ Thêm header: x-user-id, x-
  user-role, x-user-type
                                             ▼
                                        order-service (đọc header, không
  verify lại JWT)
```

Public routes (không cần token):

- `POST /api/identity/auth/*`
- `GET /api/cms/articles`, `/banners`, `/disease-categories`, `/store-config/public`
- `GET /api/catalog/products`, `/categories`, `/promotions/active`

Biến môi trường

File	Mục đích
`.env.example` (root)	`JWT_SECRET`, `SMTP_*` — dùng cho docker-compose
`backend/api-gateway/.env.example`	Chạy gateway riêng lẻ
`backend/<service>/.env.example`	Chạy từng service riêng lẻ

> **Quan trọng:** `JWT_SECRET` trong `.env` (root) phải **giống hệt** giá trị trong `backend/api-gateway/.env`.
> Docker Compose tự động đọc `.env` ở thư mục gốc.

Convention làm việc nhóm

1. **Pull trước khi code:** `git pull origin main`
2. **Tạo nhánh mới** cho mỗi tính năng: `git checkout -b feature/ten-chuc-nang`
3. Code xong → **push nhánh** → tạo Pull Request → team review → merge vào `main`
4. **KHÔNG push file `.env`** — chỉ push `.env.example`
5. Khi thay đổi database schema: cập nhật file SQL trong `infrastructure/database/` và commit để mọi người tự cập nhật

Phân công service

Thành viên	Service phụ trách
(điền tên)	identity-service
(điền tên)	catalog-service
(điền tên)	order-service

Thành viên	Service phụ trách
(điền tên)	cms-service
(điền tên)	notification-service + api-gateway

Các endpoint stub (cần implement)

Tất cả POST/PUT/DELETE hiện trả về `501 Not Implemented` kèm comment TODO hướng dẫn. Người phụ trách service chỉ cần mở file `<resource>.routes.js` và điền phần TODO là xong.

Ports summary

URL	Mô tả
`http://localhost:8000`	API Gateway (điểm vào duy nhất)
`http://localhost:8001`	identity-service (truy cập trực tiếp khi dev)
`http://localhost:8002`	catalog-service
`http://localhost:8003`	order-service
`http://localhost:8004`	cms-service
`http://localhost:8005`	notification-service
`http://localhost:3306`	MySQL 8.0 (user: root / pass: root)
`http://localhost:15672`	RabbitMQ Management UI (guest/guest)